Real-Time Schedulers for STM32: Performance Evaluation of EDF over Superloop

Vladimir Toporkov, Artem Burmyakov
Innopolis University

I. Introduction

STM32 is a family of microcontrollers developed by STMicroelectronics for embedded applications. These devices integrate a processor, memory, and peripheral interfaces on a single chip [1].

STM32-based devices support applications such as environmental monitoring [2] and equipment condition monitoring [3]. In these applications maintaining responsiveness as the workload increases is a key challenge.

The paper examines how the choice of scheduling approach affects responsiveness and processing cost on STM32. We compare two approaches: Superloop [4] and Earliest Deadline First (EDF) [5].

A. Related works

- Superloop in RTS.
- EDF and chunked EDF.
- Our evaluation.

B. Contribution

First, we compare Superloop and chunked EDF for periodic tasks with implicit deadlines on an STM32F767 microcontroller. Then, we evaluate the effects of task-set utilization on response time, deadline misses, skipped releases, and measured task-selection overhead. Also, we investigate how maximum chunk duration affects these metrics for chunked EDF. Both scheduler implementations are publicly available.

C. Paper organization

Initially, in Section II, we provide the necessary background on STM32. We describe the hardware platform in Section II-A and the project structure in Section II-B. Sections II-C and II-D describe how application functions execute and explain the differences between synthetic and sensor workloads. Next, in Section III we formalize the periodic task model used in our experiments. Finally, in Section IV, we describe Superloop, fully preemptive EDF, and chunked EDF and illustrate their schedules for the same task set.

II. Background on STM32

Let us first provide the necessary preliminaries on the hardware components, which we later use in our evaluation for schedulers.

A. Hardware platform

The central processing unit in our hardware system is an STM32 microcontroller, as illustrated in Fig. 1. **1.AB: To describe relevant details about STM32 microcontroller. To include:**

- How many microcontrollers? Uniprocessor or multiprocess system?

In turn, to model the workload to be processed by the STM32 microcontroller, we use several peripheral sensors, which are summarized in Table I. For every sensor,

- it is directly connected to the STM32 microcontroller by using some of the hardware connection interfaces from the ones listed in Table I;
- it is associated with some specialized task, which generates an infinite sequence of periodic jobs to be processed by the microcontroller (more formal details are in Section III);
- these sensors' tasks are to be executed concurrently upon the same processor, what requires the use of some scheduler.

The code that reads and processes their data forms the sensor-based workload for our scheduling experiments.

TABLE I
Sensor functions and interfaces.

Sensor	Purpose	Interface
DHT11 [6]	Temperature and humidity	Single-wire digital, GPIO
HC-SR04 [7]	Distance	Trigger/echo, GPIO
OV2640 [8]	Image acquisition	Parallel data, SCCB
SPW2430 [9]	Signal acquisition	Analog output, ADC

GPIO - General-Purpose Input/Output; SCCB - Serial Camera Control Bus; ADC - Analog-to-Digital Converter.

B. STM32 Project Structure

We use STM32CubeMX to configure the microcontroller pins and peripherals and generate a C project. We then use STM32CubeIDE to edit, build, and debug the code.

Fig. 2 shows the main project components. Application source files are stored in `Core/Src`, while header files are stored in `Core/Inc`. The `main.c` file contains system initialization and the main loop that calls application functions.

C. Default execution model

The generated `main()` function first initializes the system and peripherals, then enters an infinite `while (1)`

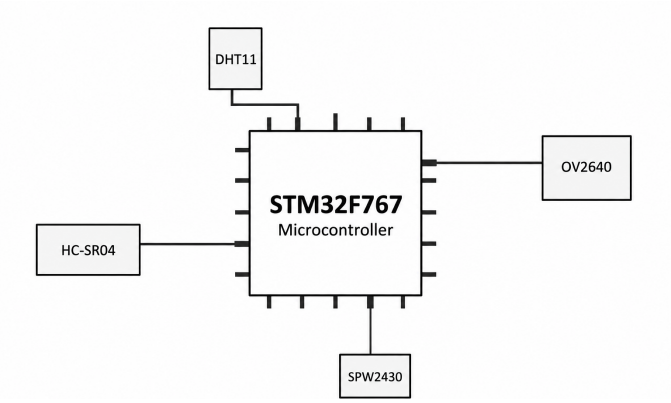


Fig. 1. Sensor platform based on STM32F767.

```
example_project/
├─ Core/ ..... Application code
│  ├─ Inc/ ..... Header files
│  └─ Src/ ..... Source files
│     ├─ main.c ..... Init. and main loop
│     ├─ stm32f7xx_it.c ..... Interrupt handlers
│     ├─ stm32f7xx_hal_msp.c ..... Low-level peripheral setup
│     └─ system_stm32f7xx.c ..... Core init. and clock state
│  └─ Startup/ ..... Startup code and interrupt vectors
├─ Drivers/ ..... Hardware support
│  └─ CMSIS/ ..... Core and MCU definitions
├─ STM32F767ZITX_FLASH.ld
├─ STM32F767ZITX_RAM.ld
└─ test_project.ioc ..... CubeMX configuration
```

Fig. 2. Main components of the STM32CubeMX-generated project.

loop (Listing 1). STM32CubeMX marks `USER CODE` regions where developers can add their own code without losing it when the project is generated again. By default, the loop contains no application functions.

Application functions can be called from inside this loop. In a Superloop, these functions run in a fixed order, and the sequence repeats after the last function returns [4] (Fig. 3). For example, `findMaximum()` runs only after `findAverage()` returns, so the execution time of one function affects when the next function can start.

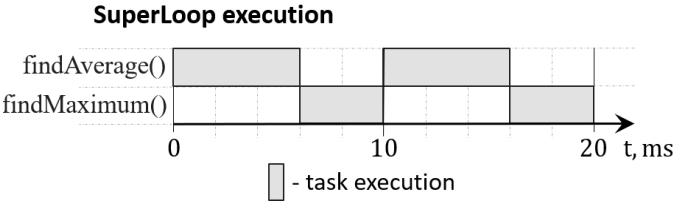


Fig. 3. Example of Superloop execution. Functions are executed sequentially in a fixed order, and the loop repeats indefinitely.

Listing 1. Simplified STM32CubeMX-generated main function showing system initialization, user-code regions, and the infinite loop.

```
int main(void) {
  /* CubeMX-generated initialization */
  MPU_Config();
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  /* Other peripheral initialization */

  /* USER CODE BEGIN 2 */
  /* User-defined application setup */
  /* USER CODE END 2 */

  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    /* User-defined functions */
  }
  /* USER CODE END 3 */
}
```

D. Application Workloads

We consider two types of application tasks: synthetic computations and sensor readings. Synthetic tasks perform calculations without using external devices. Their execution time can be adjusted to create different CPU loads. This allows us to compare schedulers under controlled conditions.

Sensor tasks read and process data from sensors connected to the STM32. Unlike synthetic tasks, they interact with real hardware. We use these tasks to evaluate the schedulers under real sensor workloads.

III. System Model

We use the periodic task model of Liu and Layland [5] and adapt the task and release notation of Burmyakov and Bini [10]. Table II summarizes the notation.

A real-time system $\mathcal{T} = \{\tau_1, \ldots, \tau_n\}$ consists of n periodic tasks executed on one core ($m = 1$). Each task $\tau_i = (C_i, D_i, P_i)$ is defined by the worst-case execution time C_i of a job, a relative deadline D_i, and a period P_i. We consider implicit deadlines, $D_i = P_i$.

Let $J_{i,j}$ denote the j-th job of τ_i, released at $r_{i,j}$. Consecutive jobs are released one period apart:

$$r_{i,j+1} = r_{i,j} + P_i. \tag{1}$$

Each job $J_{i,j}$ has an absolute deadline $d_{i,j} = r_{i,j} + D_i$. If completed, its finish time is denoted by $f_{i,j}$.

TABLE II
SYSTEM MODEL NOTATION.

Symbol	Meaning
$\mathcal{T}, n$	Task set and number of tasks
τ_i	Task i
C_i	Worst-case execution time of a job of τ_i
P_i	Task period
$D_i = P_i$	Relative deadline, equal to the period
$J_{i,j}$	The j-th job of task τ_i
$r_{i,j}$	Release time of job $J_{i,j}$
$f_{i,j}$	Job completion time
$d_{i,j} = r_{i,j} + D_i$	Absolute deadline of job $J_{i,j}$
m	Number of cores
$U_i = C_i/P_i$	Task utilization
$U_{\mathcal{T}} = \sum_{\tau_i \in \mathcal{T}} U_i$	Task-set utilization

- A schedule specifies which job executes at each time, if any.
- Preemption interrupts a job before completion to execute another job. The interrupted job may resume later.
- A completed job $J_{i,j}$ misses its deadline if $f_{i,j} > d_{i,j}$.
- The response time of a completed job $J_{i,j}$ is $f_{i,j} - r_{i,j}$.

IV. SUPERLOOP AND EDF SCHEDULERS

We next describe the Superloop and chunked EDF schedulers used in our experiments. Fully preemptive EDF serves as a reference for comparison.

A. Superloop

The Superloop scheduler has three key features:

- Periodic tasks: each task can run only after its next release time.
- Fixed order: ready tasks run in the same order.
- Non-preemptive: each job runs until it finishes.

At the start of each loop iteration, the scheduler checks all tasks using the same current time. A task that becomes ready during execution is considered in the next iteration.

Similar approaches to periodic task scheduling in a main loop are described by Koopman [11] and Jones [12]. Algorithm 1 shows this process.

Algorithm 1 Superloop Scheduler

```
1:  function SUPERLOOP(T)
2:      t_i^next ← 0,   i = 1, ..., n            ▷ Initial releases
3:      while true do                             ▷ Infinite cycle
4:          t^cur ← NOW()                         ▷ Get current system time
5:          for i ← 1 to n do
6:              ready_i ← (t_i^next ≤ t^cur)      ▷ Save readiness
7:          end for
8:          for i ← 1 to n do
9:              if ready_i then
10:                 J_{i,j} ← (τ_i, t_i^next)     ▷ Keep release time
11:                 EXECUTE(J_{i,j})
12:                 f_{i,j} ← NOW()
13:                 t_i^next ← t_i^next + P_i     ▷ Advance, then skip
14:                 while t_i^next < f_{i,j} do t_i^next ← t_i^next + P_i
15:             end if
16:         end for
17:     end while
18: end function
```

Fig. 4(a) illustrates the Superloop schedule for tasks τ_1 and τ_2. Both tasks release their first jobs at $t = 0$. Since τ_1 comes first in the polling order, $J_{1,1}$ executes until $t = 18$ ms. Job $J_{2,1}$ then starts and completes at $f_{2,1} = 28$ ms, missing its deadline $d_{2,1} = 20$ ms.

The release at $r_{2,2} = 20$ ms occurs while $J_{2,1}$ is still running. Once this job completes, the scheduler counts that release as skipped and advances the next release time to $r_{2,3} = 40$ ms. Polling continues with no ready job until then.

B. Earliest Deadline First

In contrast, fully preemptive EDF selects the ready job with the earliest absolute deadline. A newly released job preempts the running job if its deadline is earlier.

Fig. 4(b) shows the EDF schedule for the same task set. At $t = 0$, $J_{2,1}$ has the earlier deadline and executes until $t = 10$ ms. Job $J_{1,1}$ then starts, but is preempted at $r_{2,2} = 20$ ms because the new job has a deadline $d_{2,2} = 40$ ms, earlier than $d_{1,1} = 50$ ms. Job $J_{2,2}$ executes until $t = 30$ ms, after which $J_{1,1}$ resumes and completes at $t = 38$ ms. The next job of τ_2 executes from $t = 40$ to $t = 50$ ms. Thus, no deadline is missed and no release is skipped in this example.

C. Chunked Earliest Deadline First

Unlike fully preemptive EDF, our chunked EDF implementation allows switching between jobs only after an execution step finishes. Its key features are:

- Earliest deadline first: the ready job with the earliest absolute deadline is selected.
- Chunked execution: jobs run in non-preemptive steps. For computational workloads, each chunk lasts at most Δ ms.
- Repeated selection: the scheduler selects a job after each execution step.

Algorithm 2 shows this process. The Boolean flag a_i indicates whether task τ_i has an unfinished job. At each iteration, $\mathcal{R}$ contains the indices of runnable tasks. These include tasks

with due releases and active jobs that can resume. Jobs waiting for an external event are excluded.

For an active job, $d_i = r_i + D_i$, where r_i is its saved release time. Otherwise, $d_i = t_i^{\mathsf{next}} + D_i$.

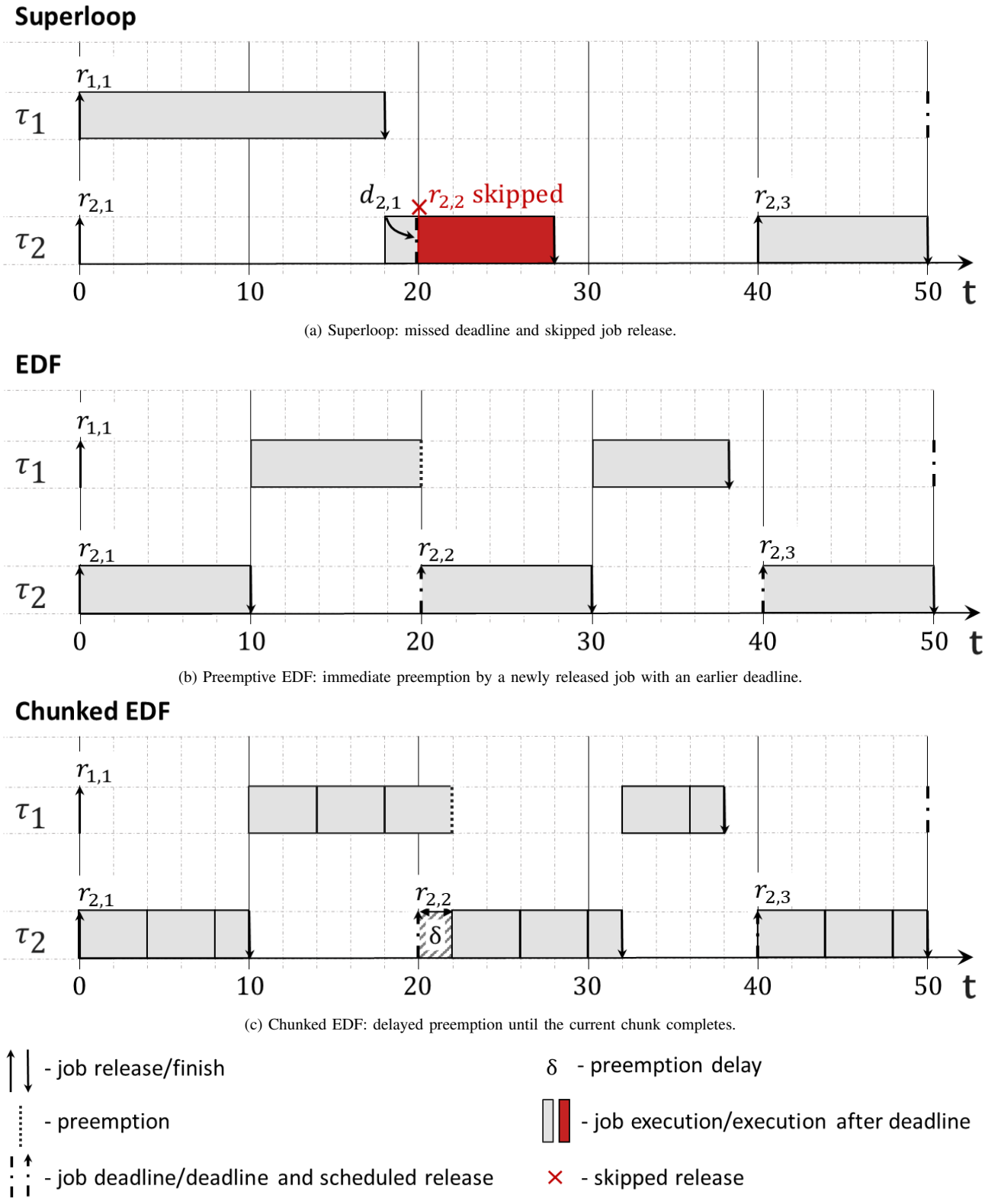
Algorithm 2 Chunked EDF Scheduler

function CHUNKEDEDF($\mathcal{T}, \Delta$)
  $t_i^{\mathsf{next}} \leftarrow 0, \quad hasActiveJob_i \leftarrow \mathsf{false}, \quad i = 1, \ldots, n$
  while true **do** ▷ Infinite cycle
    $t^{\mathsf{cur}} \leftarrow$ NOW() ▷ Get current system time
    $\mathcal{R} \leftarrow \{i \mid \text{READY}(i, t^{\mathsf{cur}})\}$ ▷ Ready task indices
    if $\mathcal{R} \neq \emptyset$ **then**
      $i^\star \leftarrow \arg\min_{i \in \mathcal{R}} d_i$ ▷ Earliest deadline
      if $\neg hasActiveJob_{i^\star}$ **then**
        $r_{i^\star} \leftarrow t_{i^\star}^{\mathsf{next}}$;
        $hasActiveJob_{i^\star} \leftarrow \mathsf{true}$ ▷ Save release
      end if
      EXECUTESTEP($i^\star, \Delta$) ▷ At most Δ
      if COMPLETED($i^\star$) **then**
        $f \leftarrow$ NOW()
        $t_{i^\star}^{\mathsf{next}} \leftarrow t_{i^\star}^{\mathsf{next}} + P_{i^\star}$ ▷ Advance, then skip
        while $t_{i^\star}^{\mathsf{next}} < f$ **do** $t_{i^\star}^{\mathsf{next}} \leftarrow t_{i^\star}^{\mathsf{next}} + P_{i^\star}$
        $hasActiveJob_{i^\star} \leftarrow \mathsf{false}$
      end if
    end if
  end while
end function

Fig. 4(c) shows the same task set with $\Delta = 4$ ms. At $r_{2,2} = 20$ ms, job $J_{1,1}$ is still executing a chunk. This chunk ends at $t = 22$ ms. Job $J_{2,2}$ then starts with a preemption delay of $\delta = 2$ ms. It executes in chunks of 4, 4, and 2 ms and completes at $t = 32$ ms. The shorter final chunk reflects the remaining work. Job $J_{1,1}$ then resumes and completes at $t = 38$ ms.

$$\tau_i = (C_i, D_i, P_i), \qquad D_i = P_i, \qquad m = 1$$
$$\tau_1 = (18, 50, 50), \qquad \tau_2 = (10, 20, 20)$$

All times in ms. Maximum chunk duration in (c): $\Delta = 4$ ms.



(a) Superloop: missed deadline and skipped job release.

(b) Preemptive EDF: immediate preemption by a newly released job with an earlier deadline.

(c) Chunked EDF: delayed preemption until the current chunk completes.

Fig. 4. Comparison of Superloop, preemptive EDF, and chunked EDF schedules for the same periodic task set.

V. Performance evaluation

A. Synthetic workload

B. Real sensors workload

References

[1] *STM32F765xx, STM32F767xx, STM32F768Ax, STM32F769xx: Datasheet*, STMicroelectronics. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32f767zi.pdf

[2] STMicroelectronics, "Process and environment monitoring," accessed: Oct. 6, 2026. [Online]. Available: https://www.st.com/en/applications/factory-automation/process-and-environment-monitoring.html

[3] ——, "Condition monitoring / predictive maintenance," accessed: Oct. 6, 2026. [Online]. Available: https://www.st.com/en/applications/factory-automation/condition-monitoring-predictive-maintenance.html

[4] Arm Keil, "Execution models: Single task system (Superloop)," accessed: Oct. 4, 2026. [Online]. Available: https://www.keil.com/pack/doc/mw/Graphic/html/gui_exec_model.html

[5] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.

[6] *DHT11 Humidity & Temperature Sensor: Technical Data Sheet*, Aosong Electronics. [Online]. Available: https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf

[7] *Ultrasonic Ranging Module HC-SR04*, Elecfreaks. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

[8] *OV2640 Datasheet*, OmniVision Technologies. [Online]. Available: https://www.uctronics.com/download/OV2640_DS.pdf

[9] *SPW2430HR5H-B: Top Port SiSonic Microphone*, Knowles Electronics. [Online]. Available: https://www.mouser.com/datasheet/2/218/SPW2430HR5H-B-1290924.pdf

[10] A. Burmyakov and E. Bini, "Improving the efficiency of global EDF exact schedulability test," *IEEE Transactions on Computers*, pp. 1–14, 2026.

[11] P. Koopman, "Multi-rate main loop tasking," Dec. 2015, accessed: Oct. 19, 2026. [Online]. Available: https://betterembsw.blogspot.com/2015/12/multi-rate-main-loop-tasking.html

[12] N. Jones. (2024) You don't need an RTOS (part 1). Accessed: Oct. 19, 2026. [Online]. Available: https://www.embeddedrelated.com/showarticle/1636.php